Knowledge Graph Mining: A Survey of Methods, Approaches, and Applications

Islam Akef Ebeid

iaebeid@utexas.edu

Abstract

In this survey, we examine Knowledge Graph mining algorithms, methods, and techniques and analyze them based on their capability to process heterogeneous knowledge graphs. First, we start with traditional graph analytics, including similarity and proximity metrics. Then we discuss the latest in deep and representation learning algorithms and methods. Following, we describe an experiment highlighting the applications of Knowledge Graph Mining in drug discovery. Finally, we reflect on several issues and future directions in Knowledge Graph Mining Research.

Introduction

The world is becoming more connected than ever through social networks, bibliographic networks, and many other real-world phenomena that can be modeled as a network. From molecular structures to electric power grids to bibliographic networks, all require a data structure like a graph where data and relations are equally represented (Hogan et al., 2020). On the other hand, data and information silos refer to isolated heterogeneous data sources within the same organization or community (Patel, 2019). These silos in business, healthcare, and education have hindered progression (Ganasan, 2019).

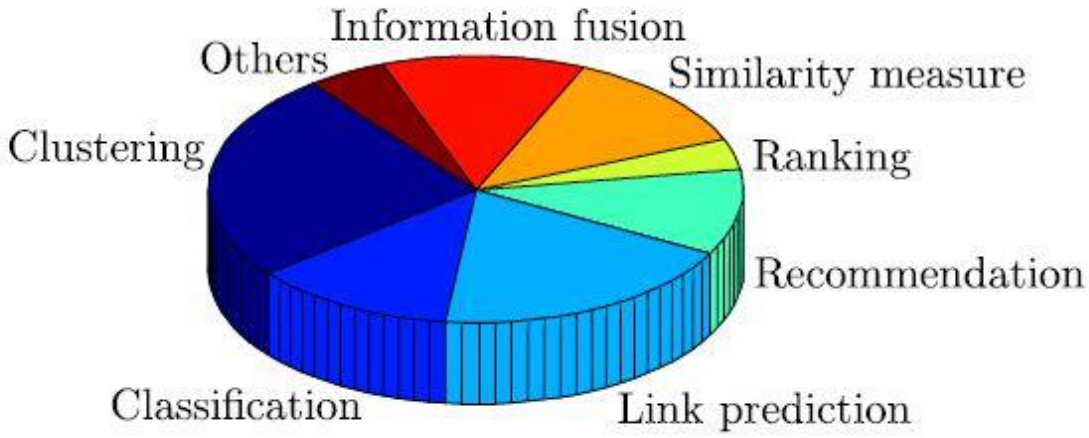
The Knowledge Graph (KG) provides a unique opportunity as an emerging model to break data and information silos through effective data integration techniques and methods (Hogan et al., 2020). With a graph representation, heterogeneous information sources can be modeled integrated, as shown and demonstrated in The Linked Open Data Cloud (The Linked Open Data Cloud, n.d.). Traditional data models, such as the relational model (Codd, 1970), provide minimal, inflexible, and computationally expensive tools to integrate heterogeneous data sources effectively. Traditional relational models rely on a predefined schema such as the Entity-Relationship (ER) model (Chen, 1976) to represent entities and properties. The relational model lacks explicit representations of relationships between entities and requires several iterations to ensure data normalization, optimize joins, and handle null values (Wickham, 2014). KG entities and relationships are represented as nodes and edges in a graph providing flexibility for further extension, integration, and knowledge discovery.

Representing relationships is crucial in many fields where relationships between entities matter most. For instance, in social networks, the connections between people are critical. In e-commerce tracking, consumer behaviors and links to products through clicking and buying are the basis for recommender systems. A KG is the backend knowledge base for reasoning and intelligent communication between machines and humans. For example, Conversational AI Agents such as Alexa, Siri, and Google Home, in addition to search engines such as Google and Others, rely on a backend KG of entities and relationships driven by a Knowledge Base such as DBPedia (Vrandečić & Krötzsch, 2014) to match queries.

The term Knowledge Graph (KG) has emerged to describe the technology that the Google search engine started using in 2012 (Paulheim, 2016). A KG is a graph representation of interrelated and semantically connected entities and relationships. KG construction can vary from manual curation to automatic construction by extracting entities and relations from unstructured text (Paulheim, 2016). KGs can be represented using the property graph model described as edges between unique nodes and properties or as RDF triples. RDF, or the Resources Descriptors Framework (Lassila et al., 1998), is a semantic representation language with a graph data model defined by the World Wide Web Consortium. The RDF triples can be used to model facts in the form of first-order logic, or <subject-predicate-object> such as <Mount Fuji, isLocatedIn, Japan> where Mount Fuji and Japan are two unique nodes or entities in a graph, and <isLocatedIn> is the relationship between the two nodes. The RDF format requires storing node names with a Unique Resource Identifier (URI). KGs can be constructed and curated under two assumptions: the open-world assumption and the closed world assumption (Cai et al., 2018). In open-world assumption, information outside the KGs is assumed to be unknown, and the KG itself is considered incomplete, such as DBpedia. In a closed world assumption, information outside the KG is considered false.

Like relational databases, KGs are graph databases queried using query languages (Hogan et al., 2020). SPARQL (Harris, n.d.) and Cypher (Cypher Query Language, n.d.) are two popular graph query languages like the Structured Query Language (SQL). SPARQL was developed as part of the semantic web technology stack to query RDF triple stores and KGs, while Cypher is a property graph query language. Due to the unstructured nature of KGs and the fact that KGs can be multigraphs, with attributes and edges that could be weighted or labeled, graph query languages are difficult and impractical for complex analysis in KGs. Those limitations have led to KG Mining methods and algorithms borrowed from graph theory, information networks, complex networks, machine learning, and geometric deep learning to adapt approaches to mine KGs more flexibly and extract deeper insights (Wang et al., 2017).

Numerous algorithms and techniques vary from traditional graph analytics and complex networks to the latest in deep geometric learning. Semantics defines knowledge graphs in many ways



Distribution of literature focusing on different data mining tasks on heterogeneous information networks. The pie chart shows that clustering or community detection, link prediction, and node classification are the research community's most talked-about data mining tasks.

Knowledge Graph Construction

Today, KG technology has been adopted in many domains and fields to store, integrate, and represent unstructured information in a structured format that is more flexible and machine-readable than the traditional ER data model (P. P. Chen, 1976). Examples of widely used and adopted KGs in different domains include Social Networks, Bibliographic and Information Networks, and Biological Networks. That includes, for example, the Facebook Social Graph (Ugander et al., 2011), Cora (McCallum et al., 2000), and Chem2Bio2RDF (B. Chen et al., 2010). Advances in research fields such as Natural Language Processing, Machine Learning (ML), Deep Learning (DL), Graph Mining, and Complex Networks powered the current proliferating landscape of KG construction and mining techniques. The techniques and methods developed by researchers in such fields have been used to mine data and information in KGs to extract insights crucial to advancing knowledge in various domains. Research in Bibliographic and Information Networks also played a role in the construction and mining of KGs, mainly through relying on traditional statistical methods and ML techniques (Sun et al., 2009).

Information networks are widely heterogeneous graphs of nodes and edges representing meta-information about a published corpus of literature such as authors, papers, publications, and venues. Hence, information networks are KGs in which graph mining techniques can be applied to extract insights about author collaboration patterns and their topics of interest. Mining information networks as KGs must lead to understanding trends such as collaboration patterns and potential drug repurposing opportunities in a specific domain without parsing the entire literature in a field (Andronis et al., 2011).

KGs traditionally have been curated manually, yet, over the years, KG construction techniques have changed. For example, Cyc (Lenat, 1995) is a general knowledge KG that was manually curated, while Freebase (Bollacker et al., 2008) and Wikidata (Vrandečić & Krötzsch, 2014), other examples of general-purpose KGs, were crowdsourced. KGs can also be extracted using NLP techniques such as in DBpedia (Lehmann et al., 2015) and YAGO (Suchanek et al., 2007). Alternatively, KGs can be constructed using a combination of manual curation and automatic extraction like in NELL (Carlson et al., 2010) and Knowledge Vault (X. Dong et al., 2014). Regardless of how a KG has been constructed, KGs need to be queried and mined to map complex real-world phenomena and eventually be utilized to solve essential research questions. For example, Facebook's Social Graph needs to be mined to suggest new friends for users. Life Sciences KGs like Chem2Bio2RDF need to be mined to answer research questions related to biomedical science in drug discovery. Moreover, Bibliographic and Information networks need to be mined to extract and infer collaboration patterns between scientists and help answer questions such as how breakthroughs in science happen?

Commonly Available Knowledge Graphs and Their Representation

Since KGs are graph representations of knowledge, we adopt different terminology when dealing with KGs compared to general graphs. Although, for example, we refer to vertices or nodes as "entities" and the edges as "relationships," we also use the terms network and graph interchangeably. In addition to the triple format, KGs can be represented in various ways, such as RDF or Adjacency Matrices. The emergence of semantic web technologies and KGs realized that traditional data modeling and database systems were too structured, rigid, and inflexible to model various real-world phenomena. Besides, relational data models never considered integrating multiple heterogeneous data sources without relying on strict data schemas or hierarchical ER diagrams. However, having flexible schemas and ontologies introduces a different set of challenges for constructing, mining, and refining KGs, as we will discuss later. Through standardization, data integration efforts have been led by the W3C by pushing the Linked Open Data (Bauer & Kaltenböck, 2012) standard through web ontology languages and knowledge representation technologies such as RDF, RDFS, and OWL. RDF, or the Resource Descriptor Framework (Lassila et al., 1998), OWL, or the Web Ontology Language (McGuinness et al., 2004), are standards and frameworks that allow and facilitate representing data as a graph of entities and relationships. Hence, they have emerged as part of the semantic web technology stack. RDF, RDFS, and OWL technologies are represented using an XML like language. These representations can be queried using a query language called SPARQL (Harris, n.d.). SPARQL is built upon pathfinding within the graph to answer a query and idea that other graph mining techniques will build upon and demonstrate later in the article.

In the following section, we will list and present several of the well-known online KGs accompanied by a supplementary table describing most of the available online KGs in Appendix A. KGs differ in many aspects and characteristics. We tried to list some of them in the table. One of the most comprehensive sources on KGs is the Linked Open Data Cloud (https://lod-cloud.net/) diagram linking most of the KG datasets that have been published in the Linked Data format with 1239 heterogeneous KGs and 16147 links between them. KGs on the Linked Open Data Cloud are subdivided into Cross-domain, Geography, Government, Life Sciences, Linguistics, Media, Publications, and Social Networks.

Datasets

Cyc (Lenat, 1995) is a cross-domain KG that relied on the manual curation of factual knowledge to power what was known as Expert Systems. OpenCyc is an online version of Cyc built in compliance with the Linked Open Data specification to be able to merge effortlessly with other KGs. Cyc can be considered a general-purpose, cross-domain knowledge graph. OpenCyc contains 2.5 million facts with 120000 instances, 45000 types, and 19000 relations (Paulheim, 2016).

Freebase (Bollacker et al., 2008) was an effort to crowdsource a cross-domain KG where anyone can edit and make changes to the existing entities, types, and relationships. Google acquired Freebase, which has not been updated since then (Paulhiem, 2016). The archived version of Freebase has 3 million facts with 27000types and 38000 relationships.

Wikidata (Vrandečić & Krötzsch, 2014), on the other hand, can be considered a cross-domain crowdsourced or a collaborative KG where it has been the data source for many Wikimedia projects, the governing organization behind Wikipedia. For example, after Google acquired Freebase, its data was moved to Wikidata (Paulheim, 2016). Wikidata contains 16 million instances with 66 million relations.

DBpedia (Lehmann et al., 2015) is a cross-domain KG created by extracting structured content from Wikipedia. DBpedia is built according to the Linked Open Data protocol, and its ontology contains 735 classes in addition to 2800 relations. It includes 4.8 million entities and 176 million triples (Paulheim, 2016).

YAGO (Suchanek et al., 2007) is a cross-domain knowledge graph extracted from Wikipedia; however, it uses WordNet as a lexical source to extract entities. YAGO is automatically constructed and relies on the fusion of several heterogeneous sources of data and information using several heuristics and measures. YAGO, in its latest release, YAGO3, has a schema that contains 488000 types and 77 relations (Hoffart et al., 2013, p. 2).

NELL (Betteridge et al., n.d.) is a cross-domain and semi-structured KG similar to YAGO and DBpedia since the construction of the KG relies on extracting information from Wikipedia. NELL, however, is an automatically curated KG. The NELL system relies on NLP approaches like the example we mentioned earlier to extract entities, types, and relationships from the web. NELL contains 2 million entities and 433000 relations, while the schema contains 425 relations.

The Google Knowledge Graph (Eder, 2012) is part of the Google Search experience, which was released in 2012 under the marketing motto "things, not strings." The whole idea was to move towards enhancing the search results and experience of the user using a window on the right side of the browser page that would show relevant, organized knowledge about the entities being searched. The knowledge captured in the "knowledge graph" window results from understanding what the user has entered as a query on a semantic level. For example, in a traditional search, when the user searches for "Benjamin Franklin," NLP techniques involving string matching would try and match the name to relevant documents and rank the order of the pages accordingly. The Google Knowledge Graph, however, would understand that Benjamin Franklin is a person that lived in Philadelphia during the eighteenth century using a Graph representation of all the information available on Benjamin Franklin on the web. A knowledge Graph is semi-structured since it relies on sources such as Wikipedia to extract and map knowledge. Knowledge Graph contains 18 billion statements, 570 million entities, and a schema of 1500 types and 35000 relations.

Google's Knowledge Vault (X. Dong et al., 2014) extends Google's Knowledge Graph and introduces a way to parse even more sources of information from the web. Knowledge Vault fuses heterogeneous sources of information like text documents, HTML tables, and structured annotations on the web. Extracted statements are then validated against Freebase. Knowledge Vault contains 45 million entities, 271 million fact statements, 1100 types, and 4500 types.

Facebook Entity Graph (Ching et al., 2015) is a social media KG that represents people as entities and their relationships. Facebook also extracts various entities like hometowns, schools, movies, bands, and books. Facebook's Entity Graph can be considered semi-structured because it pulls information from unstructured sources such as Wikipedia.

Bio2RDF (Belleau et al., 2008) is a Life Sciences KG built by mashing up and fusing existing life sciences, biological, and bioinformatics databases such as Kegg, PDB, MGI, and NCBI. The mashup was constructed to convert the data sources into a graph RDF format and then link them based on the Linked Open Data standard to create a combined KG for bioinformatics.

Chem2Bio2RDF (B. Chen et al., 2010) is an extended version of Bio2RDF. It aggregates data from multiple sources from Bio2RDF and LODD in addition to PubChem and the DrugBank. Chem2Bio2RDF contains over 13 million triples.

DBLP (Jie Tang et al., 2008) is an online bibliometric database that provides a downloadable citation network representing the relationships between authors and papers. The latest version of DBLP contains more than 4 million papers and more than 45 million citations.

Cora (McCallum et al., 2000) is a citation network containing 2708 labeled scientific publications. The network consists of 5429 links and relations. In addition to the labels for each publication, the dataset identifies which vocabulary words are missing from each publication.

Citeseer (McCallum et al., 2000) consists of 1504 machine learning publications with 2892 author references, each containing author names.

Table A. A tabular description of the most popular KGs and their properties.

	Reference	Domain	Instances	Facts	Types	Relations
OpenCyc	(Lenat, 1995)	General	118499	2413894	45153	18526
Freebase	(Bollacker et al., 2008)	General	49947845	3041722635	26507	37781
Wikidata	(Vrandečić & Krötzsch, 2014)	General	15602060	65993797	23157	1673
DBpedia	(Lehmann et al., 2015)	General	4806150	176043129	735	2813
YAGO	(Suchanek et al., 2007)	General	4595906	25946870	488469	77
NELL	(Betteridge et al., n.d.)	General	2006896	432845	285	425
The Google Knowledge Graph	(Eder, 2012)	General	570 Million	18 Billion	1500	35000
Google's Knowledge Vault	(X. Dong et al., 2014)	General	45 Million	271 Million	1100	4469

Methods in Knowledge Graph Mining

KGs are characterized by several properties that make applying mining techniques challenging. For example, KGs can operate under the assumption of an open world where all facts and relations outside the KG are considered unknown. In comparison, a closed world assumption assumes that facts outside the graph are false. KGs are also characterized by their heterogeneity, containing different types of nodes and relations and even multi relations between the same nodes. KGs can also have attributes, labels, or additional information for each node and weighted relations or edges. Besides the topology of the KG itself, those characteristics and properties should be considered whenever applying any graph analytics technique. Despite the challenging features, KGs have become the preferred data model by several research and development domains, especially AI-related fields, representing semantic knowledge in the past several years (Wilcke et al., 2017). KG mining can then be defined as a set of methods borrowed from fields like Complex Networks, Network Science, Graph Analytics, Deep Learning, and Data Structures to extract insights from the knowledge domain represented in the KG. For example, in Complex Networks, measuring a network's entropy has been one of the most important measures used when modeling molecular structures as graphs. Also, Community Detection or Node Clustering is one of the essential tasks in Network Science. Researchers in those fields developed a set of dedicated algorithms to analyze and understand communities in networks.

On the other hand, in Network Science, Node Centrality measures are essential to understand, for example, a person's popularity in a social network. In Information Science, modeling the web as a graph has been central to all search engines and information retrieval technologies. The primary task is to

come up with a measure of relevance to the results of a query to rank the results in a meaningful way. Graph Mining on Life Sciences networks such as the Protein-Protein Interaction Network (Stark et al., 2006) also has used several techniques to model, measure, and predict protein-protein interaction. In Bibliographic and Information Networks, several measures have been proposed to measure the similarity between authors to understand better collaborations (C. Yang et al., 2020). From the previous examples, we can deduce that main mining tasks and measures exist where the interpretation of the results depends on the domain knowledge represented in the graph and the nature of the research question sought through mining.

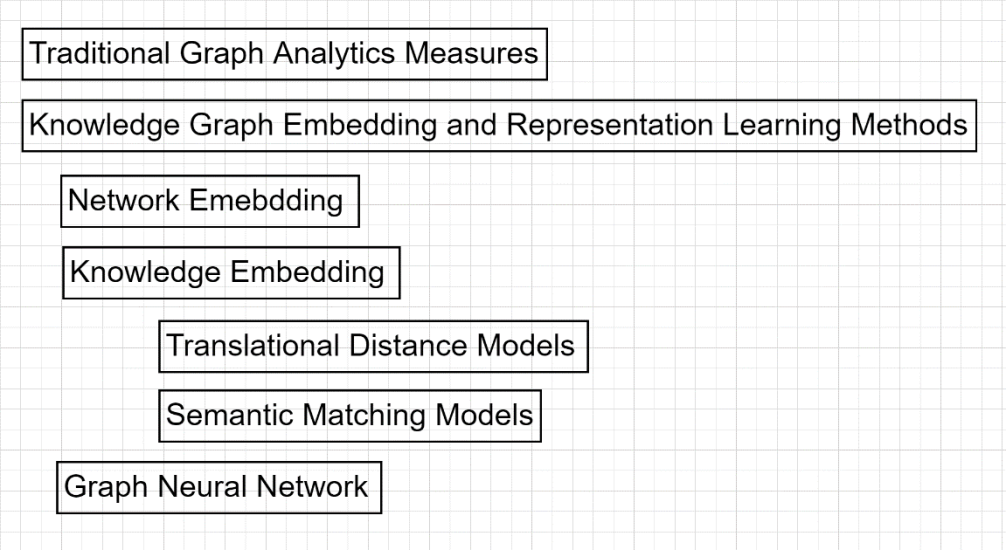


Figure 7. Taxonomy of Knowledge Graph Embedding Methods

Knowledge Graph Mining tasks

Ranking, Node Importance, and Centrality

Ranking and estimating node importance or relevance in a graph is a fundamental Graph mining task that enables many essential applications such as recommendation, search relevance, and others (Park et al., 2019). For example, information retrieval and search engine ranking provide a way to signal to the user which entities returned by the query are most relevant. In social networks, ranking might indicate which friend has a more social connection with a friend. At the same time, node importance might signal the most influential social relationships in one's group. Node importance and relevance can be measured and computed using various methods and algorithms, yet degree centrality has been one of the most common ranking nodes. Node centrality is the shortest path's average length between the node and all other nodes in the graph. Computing centrality for all the nodes in the graph can give us a way to rank the nodes in the graph accordingly. It is worth mentioning that the interpretation of a metric or a heuristic like a node centrality is dependent on the domain of the KG.

Similarity and Link Prediction

Understanding what similarity entails when comparing nodes is critical in KG mining. The traditional approach to finding similar nodes relies on comparing node neighborhoods leveraging the topology and the graph structure. Several algorithms and metrics have been developed building on different graph sampling approaches, such as random walks to measure the similarity between two objects in the graph, such as SimRank (Jeh & Widom, 2002) and PathSim (Sun et al., 2011). However, recent

developments in Graph Embedding methods have increased the similarity of tasks' accuracy. That has allowed better results in downstream machine learning mining tasks on graphs such as link prediction. In link prediction, the similarity between nodes plays an essential role as a sole indicator of the existence of a link within the graph. In a sense Link Prediction if treated as an unsupervised similarity problem between nodes cosine distance operators between nodes can predict the existence of a link within a top k search. On the other hand, link prediction is usually treated as a supervised machine learning problem on top of trained graph embedding models. The graph is split into a train, test and validation sets and a positive and negative edges are sampled from the graph. An operator is then chosen to compute an edge embedding by combining the embeddings of the two nodes forming the edge, either positive or negative. A classifier like Logistic Regression is trained to classify whether an edge exists or not based on the edge embedding vector.

Node Classification

KGs are usually characterized by having attributes and properties attached to the nodes and edges in the graph. In a heterogeneous KG, each node will likely come with a label or a type. One central task in KG mining is to predict node types for nodes that are not labeled. Machine Learning graph analytics methods have been explored previously for node and graph classification (Hogan et al., 2020). With the advancements in representation learning and graph embedding methods, node classification has been largely solved through shallow and deep learning graph models. For example, the Graph Convolutional Neural Network (GCN) described in (Kipf & Welling, 2017) is trained in a semi-supervised fashion on a node classification task. The GCN model aims to train high-quality vector representations of each node in the KG.

Node Clustering and Community Detection

Node clustering and community detection attempt to find solid and connected components within the graph. The connected components are usually indicated by the number of edges connecting the nodes within the cluster or the component. Like the popular Louvain and Leiden algorithms, several algorithms have been introduced (Traag et al., 2019) to systematically identify not just clusters of nodes based on the density of edge connections but also to solve the hard problem of community or cluster membership. The algorithms usually operate on the graph's adjacency matrix and learn the communities by reconstructing the graph. The advancements in representation learning on graphs have also fueled new techniques and methods in graph clustering and community detection.

Levels of Heterogeneity and Semantic Consideration

KGs can be large, dense, heterogeneous, and diverse. That means they can contain various node types with relationship types, multiple connections between nodes, and different attributes and properties for each node and edge. Our analysis of the graph must not only rely on either traversing and counting edges and transitions or reconstructing the topology of the graph. Our analysis must also consider as much information, attributes, and properties as possible to produce a rich, dense representation or measurements of the graph. Researchers in KG mining strive to incorporate the various semantics present in the KG in extracted, computed, or represented features. Thus, all the feature extraction algorithms and the calculated metrics presented from here and further on will be analyzed from their levels of heterogeneity and semantic consideration. For example, feature learning, embedding, and DL algorithms like Node2Vec (Grover & Leskovec, 2016) and DeepWalk (Perozzi et al., 2014), which we will introduce later, consider neither node nor edge semantics when training a model to generate node level feature vectors. Besides relevance and proximity metrics, such as PageRank (Page et al., 1999) and

SimRank (Jeh & Widom, 2002), it also does not consider the heterogeneous nature of the web as a graph.

In comparison, more recent proximity and similarity measures and algorithms consider heterogeneity, such as PathSim (Sun et al., 2011). Non-heterogenous models can be useful depending on the application, yet taking semantics into account provides more representative power to the metrics, features, or models being trained on the KG. As mentioned before, a KG can have multiple types of nodes, edges, properties, and attributes. In that sense, we will refer to algorithms and metrics that only consider the graph's topology while disregarding node types, edge types, properties, and attributes as homogenous algorithms or metrics, while algorithms and metrics that take heterogeneity into account as heterogeneous algorithms and metrics. As we will see later, each algorithm's heterogeneity levels vary widely depending on the model used and the sampling methods applied to the graph. For example, in Deep Learning approaches, shallow architectures and random walk-based methods adopted from natural language processing have had tremendous success in producing feature vectors for nodes and relations in the graph, taking into account different types of nodes and edges (Y. Dong et al., 2017).

On the other hand, Graph Neural Network-based models have not advanced enough to incorporate heterogeneous graph properties into their training (Wu et al., 2019). That might be due to several factors; first, Graph Neural Networks, such as the Graph Convolutional Network architecture introduced in (Kipf & Welling, 2017), operates on the whole graph's adjacency matrix, making it challenging to incorporate any additional semantics. Second Graph Neural Networks models are usually designed to be an end-to-end solution to a specific task on the graph. For example, the Graph Convolutional Network (Kipf & Welling, 2017) is trained on a node classification task in a semi-supervised fashion. Unlike Shallow Architectures and Random, Walk-based methods are fully unsupervised and produce more generalized graph embeddings feature vectors. Finally, even though some advancements have recently gained tract to incorporate heterogeneity into the sophisticated Graph Neural Networks architectures, such as in (Hu et al., 2020) and (Yun et al., 2020), nevertheless, these algorithms suffer from scalability issues, making them sometimes impractical to use.

On the other hand, Shallow Architectures and Random Walk based approaches adopted from Language Models (Mikolov et al., 2013) were successful due to their reliance on Heterogeneous Random Walks and Graph Sampling. This technique has been used in traditional stochastic similarity and proximity measures such as PathSim (Sun et al., 2011). In addition, researchers have adopted the idea of sampling the graph based on predefined paths from the semantic web and graph database community. For example, in RDF KGs, a graph query language is used to query the graph like SQL in relational databases. For example, SPARQL (Harris, n.d.) uses the shortest path-based approaches to determine the resultant set of the query at hand. Furthermore, the flexible graph structure where joins and null values are non-existent allows for pathfinding algorithms to extract result sets for meaningful inquiries.

Here we define the notion of levels of heterogeneity and semantic consideration for a KG mining algorithm or feature learning model. For example, we will denote a level 0 heterogeneity for an algorithm or a model, indicating that the algorithm only operates on the graph's topology. A level 1 algorithm or model means that node types are considered in the computation. A level 2 algorithm or model indicates that node attributes such as unstructured textual data are considered. Finally, a level 3 algorithm or model indicates that edge semantics are considered during training.

Finally, a level 4 algorithm indicates that attributes associated with edges are considered. Please note that heterogeneity levels do not imply the superiority of any approach, algorithm, or model over the other and are just used to distinguish different algorithms and models into classes. Algorithms, models, and approaches differ based on the application; hence no one method or model is necessarily better. Various evaluation and performance measures are usually used in KG mining to compare models and algorithms. The evaluation approaches usually demonstrate that a model or an algorithm can be better than another based on objective metrics for specific tasks, specific datasets, and using particular parameters.

Traditional Graph Analytics Measures

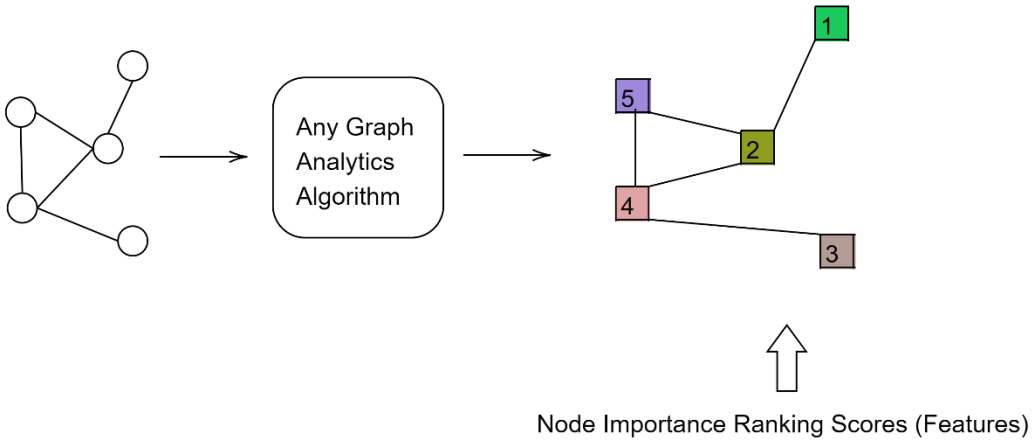


Figure 8. A symbolic representation of the role of graph analytics metrics and measures

Several real-world problems or phenomena can be represented as a graph of nodes, edges, and properties. From social networks to information networks to protein-protein interaction networks to the web to even electrical power grids, and finally to knowledge graphs. Several algorithms are developed for problems for each of these sets of specific graph types. Nevertheless, generalizing algorithms and metrics developed for particular models and graphs is common in the field. That provides a fertile ground for adopting methods for different problems. For example, in KGs, even though there is a class of feature extraction and embedding methods designed explicitly for biological networks, they can be analyzed using any of the algorithms or metrics designed for information or social networks. Besides, most existing methods are intended for general graphs and networks and are even tested and evaluated on different kinds of graphs and networks. In this article, we might use terms like Knowledge Graphs, Information Networks, The Web, and other networks interchangeably because algorithms, metrics, and methods can be used between these different types of networks and graphs freely.

Link analysis refers to measuring and analyzing the graph's connectivity to understand the similarity and proximity between nodes. That can be particularly useful in information networks where the links or the edges between the nodes represent the citations between papers. Analyzing the links in information networks could help us understand collaboration patterns between authors. It could also

help us return relevant results when searching the web when analyzing the web as a graph where pages are nodes and hyperlinks are edges between the nodes.

That quest for exploiting the graph structure of the worldwide web that is usually represented as indexed pages connected through hyperlinks in addition to the information present on each page started with the introduction of Google's PageRank (Page et al., 1999) and also the HITS algorithm (Kleinberg et al., 1999). PageRank, as an objective measure, looks at sampling the pages retrieved by a query and computing a score to reflect each node's subjective relevance or importance. Traditional methods stemming from academic search engines and formerly library sciences only relied on creating a structure out of the citation maps of each document and using simple measures such as citation counts. The HITS algorithm exploits the graph structure of the web by applying two steps: a sampling step on the retrieved "authority" pages and a weighted propagation step, which computes an "authority" score indicating the webpage's relevance. HITS is also used for community detection, one of the essential tasks in graph and information network analysis. There are three measures that the HITS algorithm relies on to return the result of a query. The measures include the Degree distribution, which suggests that a node or a page of a certain degree is proportional to a statistical distribution such as Bernoulli or Zipfan. Also, several bipartite cores and connectivity of local subgraphs are both measures related to community detection. (Page et al., 1999) The paper mentioned that their vision includes a full parsing of pages across the world wide web to extract information, entities, relationships, and contexts. This vision, however, only came to fruition with the introduction of Knowledge Graphs.

In web search and information retrieval systems, finding relevant and trustworthy pages for a query requires analyzing the links between the pages under the assumption that trustworthy pages tend to be strongly connected nodes in the graph. Accordingly, we would like to compute a score, metric, or feature for each node, indicating its relevance to a query node. The idea of computing node importance or relevance was introduced in PageRank (Page et al., 1999), the original algorithm that powered the Google Web Search Engine.

On the other hand, despite its efficiency and practicality in computing similarity and ranking within large network level 0, metrics like HITS and PageRank disregard a great deal of information in the process. Perhaps for search engines, the web is seen as a homogenous network. However, it is a primarily heterogeneous network of pages of different types and genres. In the field of bibliographic and information networks, several algorithms, metrics, and measures have been developed to compute the similarity between nodes and objects in the network. For example, we might want to find out the most similar authors in an information network. Computing a similarity score like PageRank on the authors in the network will mean that the most similar authors are the ones who are mentioned together in citations. That, however, is a rigid notion of similarity between authors in an information network in the sense that authors might be similar based on not only common papers and citations but also common venues they publish in or common topics they publish about. That has led to the need for algorithms, measures, and metrics that would consider the heterogeneity of the network, the different types of nodes/entities, and edges/relations in the network.

Here we list some of the popular traditional graph analytics algorithms and metrics used in ranking, proximity, and similarity tasks on graphs, networks in general, and KGs in particular.

PageRank (Page et al., 1999) Computing PageRank for nodes in a graph relies on first enumerating the in and out links of a given node, then computing an adjacency matrix M for the entire graph. Each node

in the graph is usually characterized by input and output links. For nodes, i and j and d_i output links the value of the adjacency matrix at $M_{ji} = 1/d_i$; otherwise, $M_{ji} = 0$. Besides, the columns of the matrix M must all sum to 1. Let vector r represent the rank vector for all the nodes in the graph and r_i is the PageRank score for a page i in vector r where:

$$r_j = \sum_{i \to j} r_i / d_i$$

In other words, if page j points to page i then we will have a non-zero entry in the matrix of scores. For all pages or nodes in our network, we can then rewrite the equation in Matrix form

$$r = M * r$$

Where M is an adjacency matrix, and r is a vector for PageRank scores for each page. Then by factorizing Matrix M, we can get a solution for these equations and find PageRank scores for each page. Note that the previous situation is a Markov Process that reaches convergences once all the pages for a query are found. The notion of a random surfer or random walker is also mentioned to describe the PageRank formulation as a method for sampling the network to find the most relevant pages to the query. It is also worth noting that PageRank can be considered a level 0 in heterogeneity as it does not assume any heterogeneity aspects of the network at hand and only considers the number of the in and out links to other nodes.

Personalized-PageRank (Lofgren et al., 2016) in PageRank, as the random walker traverses the graph to populate the matrix, a situation can occur where a page might be referring to itself loop halts the algorithm. The situation is known as the spider trap. A teleportation technique in PageRank is incorporated to break out of the spider trap while traversing the nodes. In personalized PageRank, the teleportation happens only with nodes of specific relevant topics. That can achieve level 1 heterogeneity, where the algorithm includes node types in the sampling procedure.

SimRank (Jeh & Widom, 2002) was developed to measure the similarity between nodes in the network based on proximity. For each of the two nodes under investigation, two random walkers are assigned to start from each node to find the shortest paths between the two nodes. The walkers then restart if the expected meeting distance metric is achieved, which is the expected number of steps required before the two walkers meet. SimRank is purely a distance metric even though it can operate on bipartite graphs; hence it can be considered a level 0 from the perspective of heterogeneity.

PathSim (Sun et al., 2011) introduced the meta-path-based approach to random walking in computing similarity between objects of the same type. The idea is to construct a schema of the information network at hand, representing the semantic relationships between different object/node types in the network, as shown in Figure 2.

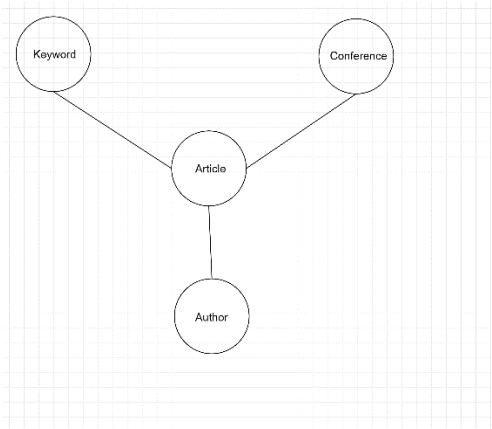


Figure 9: A meta schema of a bibliographic information network

The schema can be considered an ontology describing the network at hand. Hence a meta-path is a subschema describing a semantically meaningful path on the actual network that, if taken into consideration when computing the similarity scores, will result in a more flexible and contextual definition of which author is similar. For example, the meta-path <Author-Paper-Author> represents a direct co-authorship relationship between the two authors so that computing the PathSim score on this combination of relations in the network would be semantically meaningful and reflective of a co-authorship relation between the two authors in comparison. Besides, other similarity measures like PageRank or SimRank are biased towards nodes with high centrality. To compute the measure first, we compute a commuting matrix M for the given meta path, which is the product of the adjacency matrices for each pair of nodes in the meta path M_{ij}. Then a degree matrix is computed for each object/node in caparison M_{ii} and M_{jj}. The similarity between the two objects at hand is then calculated according to

$$s(x_i, x_j) = \frac{2M_{ij}}{M_{ii} + M_{jj}}$$

PathSim is a highly heterogeneous algorithm that can incorporate node types in meta paths describing contextually relevant relationships. Therefore, PathSim can be considered a level 3 in heterogeneity and semantic consideration.

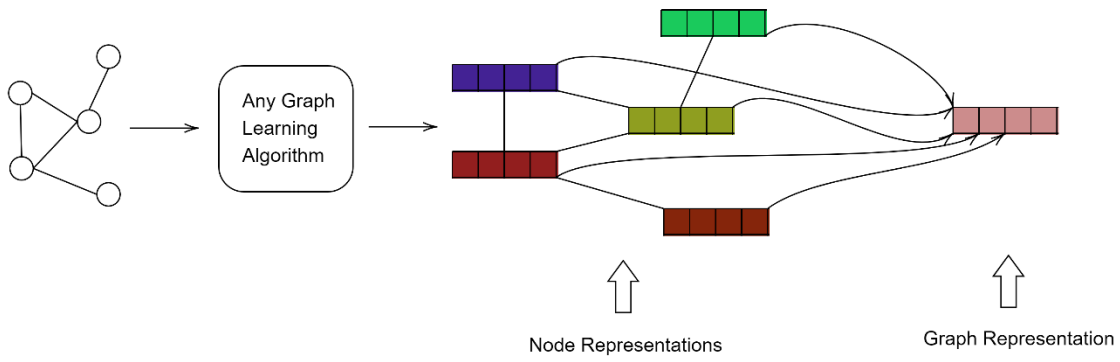


Figure 10. A symbolic representation of the role of graph embedding algorithms in learning latent feature vectors

Machine Learning is a field that aims at extracting valuable patterns from a given dataset and mapping it to a mathematical function that can predict with a certain level of confidence an unseen datapoint. Machine Learning techniques can include methods for automatically extracting features from data (Deep Learning) or relying on datasets where features are already provided. Machine Learning models also use supervised, semi-supervised and unsupervised approaches to learning mathematical functions that can map inputs to outputs. In supervised methods, datasets must be labeled with a target label. The model tries to predict the label of the provided data points iteratively over-optimizing an objective function by minimizing the error between the predicted and the expected. In unsupervised approaches, labels need not be provided. However, features will be automatically extracted, as in the case of Deep Learning or training. In addition, data will be automatically generated in the case of semi-supervised or self-supervised models.

Representation learning and deep learning methods on Graphs and Networks have gained much attention in the past decade due to their reliance on automatically extracting relevant features from the dataset without relying on stochastic and heuristic metrics and measures. Regardless of the method or the objective training function, all algorithms produce node representations in low dimensional vectors, as demonstrated in figure 3. The minimum constraint on these representations is that they preserve the graph's structure in the Euclidean vector space. The node embedding vectors learned can be passed to downstream machine learning models for classification, regression, or clustering. Alternatively, they can be used in the learning process in a semi-supervised end-to-end fashion as in Graph Neural Networks Approaches. They can also be inductive as learned for the graph's structure or transductive by forcing a scoring function.

With the success of neural networks and deep learning (unsupervised feature learning) in computer vision, imaging, and natural language processing, these methods lend themselves to graphs despite the challenges. The intention here is to embed nodes in a low-dimensional continuous vector space while preserving the structure and relations between nodes in the graph. Those learned embeddings feature vectors can be used in downstream machine learning tasks to classify nodes or predict links. Machine learning models involved in clustering and classification, such as Linear Regression or Support Vector Machines, require a structured tabular input of observations and their features. Hence, embedding a KG

in a low dimensional vector space by extension becomes crucial to advancing graph analytics to produce more robust models that can provide accurate insights into the underlying domain in the KG.

As discussed, graph mining methods, information network analysis, and complex network analysis are usually adopted to mine and analyze KGs. Representation and Deep Learning are a set of methods and techniques aimed at learning feature vectors representing and preserving the raw data's structural information and context. These representations are crucial for machine learning and deep learning tasks such as classification, prediction, and regression to be applied to KGs. As mentioned before, traditional stochastic proximity, similarity measures, and metrics sampling techniques usually lead to information loss. Representation learning methods on Graphs aim at preserving structural information as much as possible. Neural Networks and Deep Learning on KGs can be divided into 4 clusters; Matrix Factorization Methods, Language Models/Random Walk Based Methods, Graph Neural Networks, and Knowledge Graph Embedding Methods.

Network Embedding Methods

Learning distributed representations of words and phrases of language has been the focus of Natural Language Processing and Artificial Intelligence (Bengio et al., 2013) in the past decade. Advancements in research on word embedding and language feature learning have led and continue to lead breakthroughs in language modeling, fueling applications such as question answering, machine translation, and others. On the other hand, as mentioned before, prediction, reasoning, and other mining tasks on large KGs are still challenging due to KG semantics' richness and the diversity of its ontologies. Furthermore, the richness and heterogeneity of KGs make it challenging to learn features in a distributed manner that can be easily generalized to several tasks and can be transferred when the KG is updated (Q. Liu et al., 2018).

Language models offer tools, algorithms, and techniques capable of encoding any symbolic representation of any form of information, not just language. This observation has been specially made within the field of graph analytics as it was observed that the distribution of nodes in a graph usually resembles that of the frequency distribution of words in a corpus of documents (Perozzi et al., 2014). Thus language embedding models that have gained much attention in the past decade, like the Word2Vec model, which was first introduced in (Mikolov et al., 2013), lend themselves to experimentation over embedding KG nodes and relations to create high-quality node embedding vector representations. (Mikolov et al., 2013) highlights two specific techniques for training a single-layer neural network to predict words from context or the opposite. The two techniques are the Skip-gram model and the Continuous Bag of Words model and idea, which were built to extend the neural network language model (Bengio et al., 2003). They also introduced word and vector similarity techniques to evaluate the extracted word embedding quality. In the original Word2Vec, a hierarchical Softmax approach approximates the Skip Gram model's objective function applied to the text.

The Skip-gram model used in training can then be formally described given a sequence of training words $\left(w_1, w_2, \ldots, w_t\right)$ the goal is then to maximize the average probability:

$$[\frac{1}{T}\sum_{T}^{t=1}\sum_{-c\leq j\leq c, j\neq 0}\log p\left(w_{t+j}|w_t\right)]$$

Where c is the size of the training window around the current word w_tComputing the full Softmax function is impractical, so a sampling approach such as Hierarchical Softmax or Negative Sampling is needed, as described in (Mikolov et al., 2013). Negative sampling is done by reformatting the data so that the problem becomes a logistic regression-based word classification rather than a prediction problem. The input is a context window of words where each word is classified as 1 if it exists within the context. Otherwise, 0 if the word was negatively sampled outside the context window. We then introduce negative samples to the input data, where we randomly sample words that do not exist in the context of the current word so that they are classified as 0. We then approximate the Softmax function by maximizing the log probability of the Softmax:

$$[\log \sigma\left(v'^{T}_{w_O} v_{w_r}\right) + \sum_{i=1}^{k} \mathbb{E}_{w_t \sim P_n(w)} \left[\log \sigma\left(-v'^{T}_{w_O} v_{w_r}\right)\right]]$$

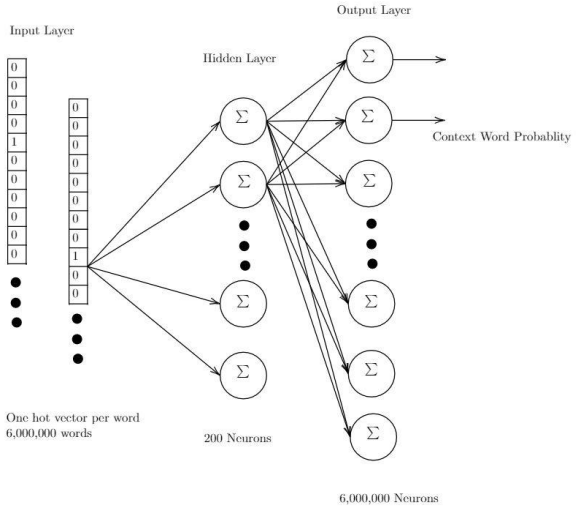


Figure 6. Word2vec Skip Gram model where each word in the vocabulary is fed to the network in the form of a one-hot vector representation. The hidden layer consists of 200 neurons. Note the number of neurons is a hyperparameter. After training the hidden layer weights using Skip-Gram and Negative Sampling, the neurons' weights will be extracted, representing the embedding vector for each word in the vocabulary.

Similarly, in Random Walk-based graph embedding methods, neural network-based language models like Word2Vec (Mikolov et al., 2013) have adopted a two-step approach to learning embedding vectors for nodes in the graph. The first step relies on sampling the graph based on a random walk strategy instead of operating on the entire graph in the form of an adjacency matrix. The random walk strategy is the crux to incorporating as much heterogeneity as possible to be considered during the unsupervised training approach that the neural network language model uses. For example, in Word2Vec, the text is first tokenized into sentences; similarly, the random walks mimic the process of creating meaningful "sentences" out of the graph as a sampling approach.

Several recent graph embedding algorithms have proposed Word2Vec-based network representation learning frameworks such as DeepWalk, LINE, Node2Vec, Edge2Vec, and Metapath2Vec. Instead of handcrafted network feature design, these representation learning methods enable the automatic discovery of useful and meaningful latent features from the raw networks (Q. Wang et al., 2017). Next, we list the most popular Random Walk based Graph Embedding algorithms used to embed KGs.

DeepWalk (Perozzi et al., 2014) proposed to sample the graph based on short random walks. The produced corpus is then used to train a Skip-gram version of the Word2Vec model, as described in (Mikolov et al., 2013). The training is done assuming that the node distributions in a graph are similar to the word distribution in a text document in natural language.

LINE (Jian Tang, Qu, Wang, et al., 2015) Attempts to embed a homogenous network by preserving the network's second-order proximity under the assumption that similar nodes have similar neighborhoods and thus should be represented closer to each other in the vector space.

PTE (Jian Tang, Qu, & Mei, 2015) Originally developed as a language model, PTE or Predictive Text Embedding is naturally adapted to networks as it attempts to embed a co-occurrence network of words, labels, and documents into a low dimensional vector space in a semi-supervised manner. PTE combines the simplicity of the unsupervised and generalized Skip-gram model (Mikolov et al., 2013) and the depth of leveraging labeled data of Convolutional Neural Networks in Language Modeling (Kim, 2014). A softmax function approximates the probability of nodes i and j having an edge between them then the second-order proximity by minimizing the objective:

$$O = \sum_{j \in B} \lambda_j d(\hat{p}(\cdot|v_j), p(\cdot|v_j))$$

Where d is the KL divergence between the conditional probability distribution and the estimated one, lambda is an important label.

Node2Vec (Grover & Leskovec, 2016) was introduced to build on the 2 step approach as DeepWalk, where the graph is sampled based on a random walk strategy, then passing the sampled corpus to a word2vec model where Skip-gram with negative sampling is applied to the sampled graph. Next, Node2vec expanded the random walk to include second-order neighborhoods to sample the graph, where they introduced two parameters to balance between a depth-first walk and a breadth-first walk. The random walk strategy has strengthened the sampled graph's proximity preserving properties, leading to higher quality node embeddings.

Nevertheless, the previous approaches have not considered any aspects of heterogeneity in the graph and have relied on preserving the proximity of nodes and the topology of the graph; hence the previous algorithms can be considered level 0 in heterogeneity and semantic consideration.

Edge2Vec (Gao et al., 2018) introduces a level 2 approach to sampling the graph based on a precomputed edge-type transition matrix to represent the network heterogeneity. The transition matrix is trained using an Expectation Maximation Algorithm. The transition matrix is then used to construct a training corpus to train node embeddings using a Skip-gram model.

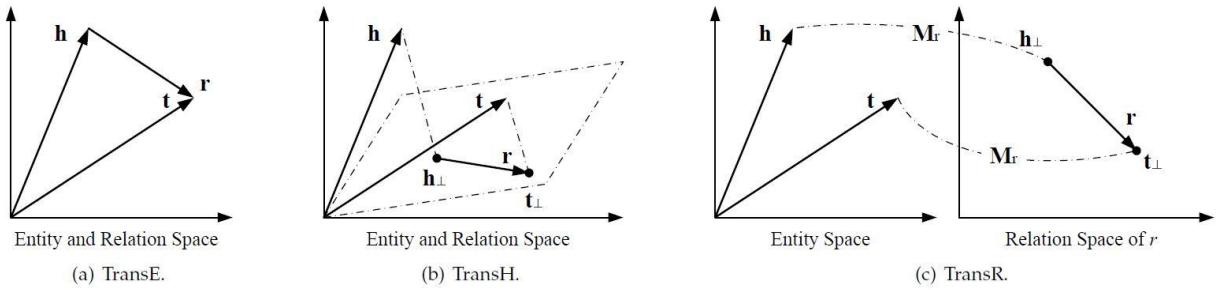
Metapath2Vec (Y. Dong et al., 2017) in metapath2vec, a level 2 approach, is also used to sample the graph guided by metapaths, as explained before in PathSim (Sun et al., 2011). The generated meta path guided corpus of sampled nodes is then used to train a heterogenous Skip-gram model where the Negative Sampling is performed heterogeneously where the negative samples are drawn from nodes of the same type.

Knowledge Embedding Methods

Statistical Relational Learning for KGs represents a different set of techniques compared to the previous structural preserving graph embedding algorithms, which vary regarding heterogeneity. Fact embedding relies on a different model where the graph is seen as a set of triplets consisting of a head (h), a relation (r), and a tail (t). For example, a triplet or a fact such as <The Stonehenge, isLocatedIn, England> where the head entity is the Stonehenge, the relation is isLocatedIn, and the tail entity is the country of England, the aim is to compute low dimensional vectors for each entity and relation. Relational embedding methods can be categorized into two classes: Translational Distance Models and Semantic Matching Models (Q. Wang et al., 2017).

Translational Distance Methods

The technique relies on enforcing a scoring function on the triplet vector representation; eventually, the vector representations will be learned over many iterations. In more detail, entities are usually represented as point vectors in the Euclidean space, and relations are represented as operations on these vectors represented as vector or matrix transformations. Then the scoring function usually is designed to enforce a specific transformation of head and tail entity representations to measure the plausibility of the fact triplet. Despite their drawbacks in dealing with metarelational data, Translational Distance Methods can be considered level 3 heterogeneous due to their focus on embedding relationships between entities. Next, we list some of the most popular algorithms in this category.



TransE (Bordes et al., 2013) learns to represent entities and relations in the vector space. The relation vector is a translation vector between the head entity and the tail entity (h,r,t). The objective is to learn vector relation vector r that minimizes the distance error h+r=t when (h,r,t) is a fact. The idea also originates from the evaluation procedure of word embedding models introduced (Mikolov et al., 2013). In Word2Vec, the operators on word embedding vectors have an analogy property describing relationships between words. For example, the relationship <isCapitalOf> can be evaluated on word embedding as Berlin + Germany = Paris + France. The resulting vectors on both sides of the equation will be very similar. Thus, TransE enforces the negative distance between h + r and t, where the score is expected to be large when the fact is true.

$$f_r(h,t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_{1/2}$$

TransE, however, fails in dealing with one too many and many too many relationships as in what if the same node has multiple edges with the same node? This means that the node will have learned two different node embedding vectors, a feat that TransE is not designed to handle.

TransH (Z. Wang et al., 2014) TransH expands on TransE by projecting each relation to a hyperplane with a normal vector W_r allowing each node to have multiple embedding vectors depending on the relation or the hyperplane their relation is projected on. The head and the tail embedding vectors are first projected on the hyperplane then the scoring function becomes the maximum of the negative distance between the project head vector and tail vectors:

$$\mathbf{h}_\perp = \mathbf{h} - \mathbf{w}_r^\top \mathbf{h}\mathbf{w}_r, \quad \mathbf{t}_\perp = \mathbf{t} - \mathbf{w}_r^\top \mathbf{t}\mathbf{w}_r$$

$$f_r(h,t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2$$

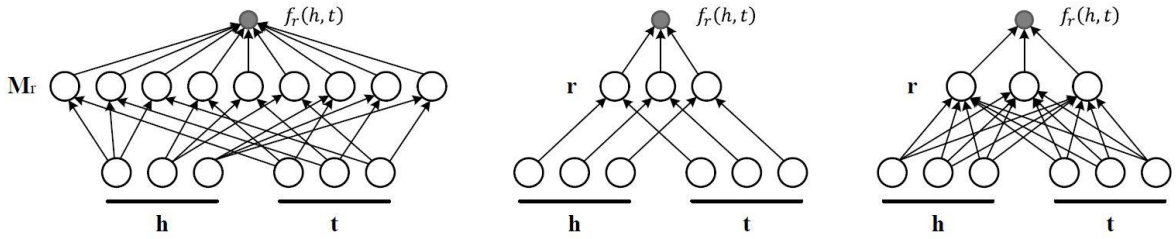
TransR (Lin et al., 2015) separates the entity and the relation into two distinct vector spaces rather than projecting each relation vector to a hyperplane. The objective is to learn a translation matrix that maps the two spaces.

$$\mathbf{h}_\perp = \mathbf{M}_r\mathbf{h}, \quad \mathbf{t}_\perp = \mathbf{M}_r\mathbf{t}$$

$$f_r(h,t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|_2^2$$

TransR is very apt to model many relationships, yet the computation of the translation matrix M is quite expensive.

Semantic Matching Based Methods



(a) RESCAL. (b) DistMult. (c) HolE.

RESCAL (Jenatton et al., 2012) uses a tensor factorization technique to model the fact triples as a 3D tensor. Each relation is represented as a matrix modeling pairwise vector interactions between the two entities.

$$f_r(h,t) = \mathbf{h}^\top \mathbf{M}_r \mathbf{t} = \sum_{i=0}^{d-1}\sum_{j=0}^{d-1} [\mathbf{M}_r]_{ij} \cdot [\mathbf{h}]_i \cdot [\mathbf{t}]_j$$

The objective is to maximize the score of the relation matrix when the fact triples hold.

DistMult (B. Yang et al., 2015) restricts the relationships matrix in RESCAL to a diagonal matrix where the dimension of the matrix becomes similar to the dimension of the head and tail vector for more straightforward computations.

$$f_r(h,t) = \mathbf{h}^\top \mathrm{diag}(\mathbf{r})\mathbf{t} = \sum_{i=0}^{d-1} [\mathbf{r}]_i \cdot [\mathbf{h}]_i \cdot [\mathbf{t}]_i.$$

DistMult, however, is only capable of dealing with symmetric relations only.

HoLE (Nickel et al., 2016) drops the idea of a relation matrix altogether. It represents the relation as a vector and scores the fact based on a circular correlation between the head and the tail entity vector and their interaction with the relation vector.

$$f_r(h,t) = \mathbf{r}^\top (\mathbf{h} \star \mathbf{t})$$

The circular correlation operator used in HoLE allows for modeling asymmetric relationships between entities.

Graph Neural Networks Methods

Graph Neural Network models rely on the graph's structure to learn numerical representations of nodes on specific supervised tasks while aggregating operators and messages passing over the neighborhood of each node. From a heterogeneity standpoint, Graph, Neural Network Models pioneered in (Scarselli et al., 2009) and (Micheli 2009) were never designed to accommodate semantics and diverse properties in a heterogeneous graph. Only recent efforts have started to incorporate graph heterogeneity into more sophisticated Graph Neural Network architectures, such as in the Heterogeneous Graph Transformer Model (Hu et al., 2020). Graph Neural Networks, according to (Wu et al., 2019), can be divided into (RecGNN, ConvGNN[Spectral, Spatial], Graph Autoencoder [learn network embedding, generate new graph], Spatial-temporal Graph neural network). The techniques employed in Graph neural networks are diverse. Methods and techniques include using LSTM (Hamilton et al., 2017), Self-attention aggregation, and evaluation methods (Veličković et al., 2017). These methods can also be specialized for specific tasks such as malicious account detection (Z. Liu et al., 2018).

Graph Convolutional Networks (GCN) (Kipf & Welling, 2017) revived and rejuvenated research in Graph Neural Networks and demonstrated that a semi-supervised graph neural network training that relies on applying simple convolutional operators to aggregate neighbor information for each node in the graph is capable of producing high-quality node embedding as compared to random walk based methods such as DeepWalk.

GraphSAGE (Hamilton et al., 2017) is an inductive GNN-based approach to solving the problem of what if, after we trained an embedding model for the existing graph, a new node comes in? Previous node embedding algorithms rely on a transductive approach to generate node embedding by training a model that would leverage the graph structure and topology information. GraphSAGE offers a way to leverage the existing node embedding to inductively produce embedding feature vectors for newly introduced nodes by aligning the new observed nodes to the existing trained embedding space. The main contribution of GraphSAGE is a set of flexible neighbor aggregators.

Discussion

Applications of Knowledge Graph Mining in Biomedical Science and Data-Driven Drug Discovery

Research in Biomedicine has always been an information-driven endeavor; a data-driven approach is needed to aid knowledge discovery processes in biomedicine. Mining bibliographic databases and information networks for latent knowledge have always been central in knowledge discovery. The idea is that latent knowledge ready to be discovered is always reliant on parsing large amounts of literature and coming up with new ideas, hypotheses, and theories. A significant body of research has explored the automatic extraction of knowledge from the biomedical scientific literature (Tshitoyan et al., 2019). Recent work has pointed towards the potential of applying Knowledge Graph driven Knowledge Discovery to the biomedical scientific literature (Jian Xu et al., 2020). By applying the latest in Natural Language Models, we can extract distinguishable entities and nouns from free text to aid in Knowledge Graph construction and Mining (J. Lee et al., 2020). Drugs, diseases, and genes are central elements in biomedicine, whereby understating how the three bio entities are related to each other, we tap into the potential of the automatic discovery of cures to hard sought diseases like Cancer.
The expert finding is the process of quickly identifying experts within a domain of knowledge in a social, organizational, or a bibliography network (Serdyukov et al., 2008). Besides, understanding and finding ways to quickly and efficiently finding experts for specific bio entities is a process that is highly sought in biomedical literature mining due to its potential in bridging the gaps of translational research and of automating metanalysis of studies which is a hugely time-consuming process. Expert finding systems in information and bibliography networks often rely on community detection methods. Knowledge Graph Embedding Methods have not been testing with direct application to expert-finding as a similarity and proximity problem.

The importance of Knowledge Graphs as a backbone for AI from an Information Science Perspective

In Information Science, the DIKW model (Ackoff, 1989) describes how a transformation happens from raw data to wisdom. Information is described in terms of data and knowledge in words of information and wisdom in terms of knowledge. That is that our process of extracting insights from data that would eventually transform into wisdom is mediated by knowledge. Acquiring knowledge can be thought of as the individual internal process where the person acquires information and stores it as knowledge through interpretation and perception. Another way to think of the DIKW model is that it describes the process by which what is objective transforms into what is subjective, leading to wise decision-making (Zins, 2007). In that sense, data and information operate on an objective level where information, according to (Floridi, 2005), can be categorized into 4 categories. First, information about something such as a flight timetable at an airport. Second, information can also be as something, such as fingerprints and genomes. Third, information can be for something, such as instructions and manuals. Finally, information can be in something such as patterns and biological processes.

On the other hand, prominent researchers in information theory, such as Claude Shannon in (Shannon, 1953), saw information from a purely quantitative and objective perspective. Though the quantitative perspective on information is useful in materializing information as systems aimed towards conveying knowledge and wisdom, it does not describe the process by which information is converted to knowledge subjectively on the receiver side. Another way of defining information, according to (Floridi,

2005), is that it is the genuinely factual semantic content of well-formed data. However, it is no big secret that information science theory and definitions have not been well defined or understood. That is not necessarily due to any lack in the efforts made by Philosophy of Information scholars but because the nature of the information itself dictates the lack of proper theory. Information is hard to define because it is hard to define information in terms of itself, which is in words. Another view of information (Floridi, 2005) also describes the transformation of data into information through well formation and meaning. However, the previous definition merges what is objective with what is subjective such as meaning. Information sources like Encyclopedias and Knowledge Bases, which act as wells of collected data and facts presented and distilled as information, turn to knowledge upon receipt by the reader. The knowledge part is the receiver or the readers' role in transforming the information they received through interpretation and inference into knowledge and eventually acted upon the knowledge in wisdom. One of the goals of AI technologies is to unify and evolve computing from a mere tool used to crunch numbers into a reconstruction of the human ability to distill knowledge and wisdom from data and information. Providing knowledge level capabilities to computers is not a simple task as a need to represent real-world semantic concepts arise. Several technological advances are needed in terms of data storage, computing power, and methods to create and represent knowledge on a machine. The computer needs to understand concepts to bridge the gap between data and information. Through mining, the instances of the concepts, i.e., the data and the information, knowledge can be reconstructed by a machine. Hence the role of research in KG mining has transcended traditional needs for more efficient data storage to a crucial tool for providing machines with the capability of understanding our complex world.

Significant hurdles and future directions in Knowledge Graph Mining research

Heterogeneity & Semantics in Knowledge Graph Mining

Since the advent of Expert Systems powered by Knowledge Bases, that understanding of knowledge has been known as deductive and precise logical and follows the facts in the Knowledge Base based on Ontology. On the other hand, advancements in Machine Learning and Pattern Recognition, a field that aims at "learning" features in a dataset as opposed to traditional statistics where features have to be predefined. Therefore, inductive Knowledge has been one of the goals sought in KG mining. That is extracting deep, valuable insights that are not necessarily as accurate as Reasoning and Deductive methods but more insightful (Hogan et al., 2020).

References

Ackoff, R. L. (1989). From data to wisdom. *Journal of Applied Systems Analysis*, *16*(1), 3–9.

Andronis, C., Sharma, A., Virvilis, V., Deftereos, S., & Persidis, A. (2011). Literature mining, ontologies, and information visualization for drug repurposing. *Briefings in Bioinformatics*, *12*(4), 357–368. https://doi.org/10.1093/bib/bbr005

Bachman, W. (1969). Data structure diagrams. *DataBase*, 4–10.

Bauer, F., & Kaltenböck, M. (2012). *Linked open data: The essentials : a quick start guide for decision makers.*

Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., & Morissette, J. (2008). Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics*, *41*(5), 706–716. https://doi.org/10.1016/j.jbi.2008.03.004

Bengio, Y., Courville, A., & Vincent, P. (2013). Representation Learning: A Review and New Perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *35*(8), 1798–1828. https://doi.org/10.1109/TPAMI.2013.50

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, *3*(Feb), 1137–1155. http://www.jmlr.org/papers/v3/bengio03a.html?source=post_page----------

Berners-Lee, T. (1992). *Worldwide Web: The Information Universe. 1*, 7.

Betteridge, J., Carlson, A., Hong, S. A., Jr, E. R. H., Law, E. L. M., Mitchell, T. M., & Wang, S. H. (n.d.). *Toward Never Ending Language Learning*. 2.

Bollacker, K. D., Evans, C., Paritosh, P., Sturge, T., & Taylor, J. (2008). Freebase: A collaboratively created graph database for structuring human knowledge. *SIGMOD Conference*. https://doi.org/10.1145/1376616.1376746

Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., & Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. *Advances in Neural Information Processing Systems*, 2787–2795.

Carlson, A., Betteridge, J., Kisiel, B., Settles, B., Hruschka, E. R., & Mitchell, T. M. (2010). Toward an architecture for never-ending language learning. *Twenty-Fourth AAAI Conference on Artificial Intelligence*.

Chen, B., Dong, X., Jiao, D., Wang, H., Zhu, Q., Ding, Y., & Wild, D. J. (2010). Chem2Bio2RDF: A semantic framework for linking and data mining chemogenomic and systems chemical biology data. *BMC Bioinformatics*, *11*(1), 255. https://doi.org/10.1186/1471-2105-11-255

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*.

Ching, A., Edunov, S., Kabiljo, M., Logothetis, D., & Muthukrishnan, S. (2015). One trillion edges: Graph processing at Facebook-scale. *Proceedings of the VLDB Endowment*, *8*(12), 1804–1815. https://doi.org/10.14778/2824032.2824077

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377–387. https://doi.org/10.1145/362384.362685

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *ArXiv:1810.04805 [Cs]*. http://arxiv.org/abs/1810.04805

Dong, X., Gabrilovich, E., Heitz, G., Horn, W., Lao, N., Murphy, K., Strohmann, T., Sun, S., & Zhang, W. (2014). Knowledge Vault: A Web-scale Approach to Probabilistic Knowledge Fusion. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 601–610. https://doi.org/10.1145/2623330.2623623

Dong, Y., Chawla, N. V., & Swami, A. (2017). Metapath2Vec: Scalable Representation Learning for Heterogeneous Networks. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 135–144. https://doi.org/10.1145/3097983.3098036

Eder, J. S. (2012). *Knowledge graph based search system* (United States Patent No. US20120158633A1). https://patents.google.com/patent/US20120158633A1/en

Floridi, L. (2005). *Semantic Conceptions of Information*. https://seop.illc.uva.nl/entries/information-semantic/

Gao, Z., Fu, G., Ouyang, C., Tsutsui, S., Liu, X., Yang, J., Gessner, C., Foote, B., Wild, D., Yu, Q., & others. (2018). edge2vec: Representation learning using edge semantics for biomedical knowledge discovery. *ArXiv Preprint ArXiv:1809.02269*.

Grover, A., & Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16*, 855–864. https://doi.org/10.1145/2939672.2939754

Gruber, T. R. (1995). Toward principles for the design of ontologies used for knowledge sharing? *International Journal of Human-Computer Studies*, *43*(5), 907–928. https://doi.org/10.1006/ijhc.1995.1081

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 1024–1034.

Harris, S. (n.d.). *SPARQL 1.1 Query Language*. Retrieved July 27, 2020, from https://www.w3.org/TR/sparql11-query/

Hoffart, J., Suchanek, F. M., Berberich, K., & Weikum, G. (2013). YAGO2: A spatially and temporally enhanced knowledge base from Wikipedia. *Artificial Intelligence*, *194*, 28–61. https://doi.org/10.1016/j.artint.2012.06.001

Hogan, A., Blomqvist, E., & Cochez, M. (2020). *Knowledge Graphs*. 131.

Hu, Z., Dong, Y., Wang, K., & Sun, Y. (2020). Heterogeneous Graph Transformer. *Proceedings of The Web Conference 2020*, 2704–2710. https://doi.org/10.1145/3366423.3380027

Jeh, G., & Widom, J. (2002). SimRank: A measure of structural-context similarity. *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 538–543.

Jenatton, R., Roux, N. L., Bordes, A., & Obozinski, G. R. (2012). A latent factor model for highly multi-relational data. In F. Pereira, C. J. C. Burges, L. Bottou, & K. Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems 25* (pp. 3167–3175). Curran Associates, Inc. http://papers.nips.cc/paper/4744-a-latent-factor-model-for-highly-multi-relational-data.pdf

Jian Xu, Ying Ding, Sunkyu Kim, Minbyul Jeong, Donghyeon Kim, Jaewoo Kang, Justin Rousseau, Xin Li, Weijia Xu, vetle torvik, Min Song, & Daifeng Li. (2020). Building PubMed Knowledge Graph. *Scientific Data*.

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *ArXiv:1408.5882 [Cs]*. http://arxiv.org/abs/1408.5882

Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *ArXiv:1609.02907 [Cs, Stat]*. http://arxiv.org/abs/1609.02907

Kleinberg, J. M., Kumar, R., Raghavan, P., Rajagopalan, S., & Tomkins, A. (1999). The Web as a Graph: Measurements, Models, and Methods. *COCOON*. https://doi.org/10.1007/3-540-48686-0_1

Lassila, O., Swick, R. R., & others. (1998). *Resource description framework (RDF) model and syntax specification*.

Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: A pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, *36*(4), 1234–1240. https://doi.org/10.1093/bioinformatics/btz682

Lee, Y., Ke, H., Yen, T., Huang, H., & Chen, H. (2019). Combining and learning word embedding with WordNet for semantic relatedness and similarity measurement. *Journal of the Association for Information Science and Technology*, asi.24289. https://doi.org/10.1002/asi.24289

Lehmann, J., Isele, R., Jakob, M., Jentzsch, A., Kontokostas, D., Mendes, P. N., Hellmann, S., Morsey, M., Van Kleef, P., Auer, S., & others. (2015). DBpedia–a large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web*, *6*(2), 167–195.

Lenat, D. B. (1995). CYC: A large-scale investment in knowledge infrastructure. *Communications of the ACM*, *38*(11), 33–38. https://doi.org/10.1145/219717.219745

Lin, Y., Liu, Z., Sun, M., Liu, Y., & Zhu, X. (2015, February 19). Learning Entity and Relation Embeddings for Knowledge Graph Completion. *Twenty-Ninth AAAI Conference on Artificial Intelligence*. Twenty-Ninth AAAI Conference on Artificial Intelligence. https://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9571

Liu, Q., Wan, R., Yang, X., Zeng, Y., & Zhang, H. (2018). *Generalized Embedding Model for Knowledge Graph Mining*.

Liu, Z., Chen, C., Yang, X., Zhou, J., Li, X., & Song, L. (2018). Heterogeneous Graph Neural Networks for Malicious Account Detection. *Proceedings of the 27th ACM International Conference on Information and Knowledge Management - CIKM '18*, 2077–2085. https://doi.org/10.1145/3269206.3272010

Lofgren, P., Banerjee, S., & Goel, A. (2016). Personalized PageRank Estimation and Search: A Bidirectional Approach. *Proceedings of the Ninth ACM International Conference on Web Search and Data Mining - WSDM '16*, 163–172. https://doi.org/10.1145/2835776.2835823

McCallum, A. K., Nigam, K., Rennie, J., & Seymore, K. (2000). Automating the Construction of Internet Portals with Machine Learning. *Information Retrieval*, *3*(2), 127–163. https://doi.org/10.1023/A:1009953814988

McGuinness, D. L., Van Harmelen, F., & others. (2004). OWL web ontology language overview. *W3C Recommendation*, *10*(10), 2004.

Micheli, A. (2009). Neural Network for Graphs: A Contextual Constructive Approach. *IEEE Transactions on Neural Networks*, *20*(3), 498–511. https://doi.org/10.1109/TNN.2008.2010350

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 3111–3119.

Nadeau, D., & Sekine, S. (2007). A survey of named entity recognition and classification. *Lingvisticae Investigationes*, *30*(1), 3–26.

Nickel, M., Rosasco, L., & Poggio, T. (2016). Holographic embeddings of knowledge graphs. *Thirtieth AAAI Conference on Artificial Intelligence*.

Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank citation ranking: Bringing order to the web.*

Park, N., Kan, A., Dong, X. L., Zhao, T., & Faloutsos, C. (2019). Estimating Node Importance in Knowledge Graphs Using Graph Neural Networks. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 596–606. https://doi.org/10.1145/3292500.3330855

Paulheim, H. (2016). Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, *8*(3), 489–508. https://doi.org/10.3233/SW-160218

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 701–710. https://doi.org/10.1145/2623330.2623732

Popescu, I. (1992). A relational model for knowledge representation in expert systems. *Journal of Systems and Software*, *18*(2), 147–155. https://doi.org/10.1016/0164-1212(92)90124-3

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). The Graph Neural Network Model. *IEEE Transactions on Neural Networks*, *20*(1), 61–80. https://doi.org/10.1109/TNN.2008.2005605

Senko, M. E., Altman, E. B., Astrahan, M. M., & Fehder, P. L. (1973). Data structures and accessing in database systems, I: Evolution of information systems. *IBM Systems Journal*, *12*(1), 30–44. https://doi.org/10.1147/sj.121.0030

Serdyukov, P., Rode, H., & Hiemstra, D. (2008). Modeling multi-step relevance propagation for expert finding. *Proceeding of the 17th ACM Conference on Information and Knowledge Mining - CIKM '08*, 1133. https://doi.org/10.1145/1458082.1458232

Settles, B. (2004). Biomedical Named Entity Recognition using Conditional Random Fields and Rich Feature Sets. *Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and Its Applications (NLPBA/BioNLP)*, 107–110. https://www.aclweb.org/anthology/W04-1221

Shannon, C. (1953). The lattice theory of information. *Transactions of the IRE Professional Group on Information Theory*, *1*(1), 105–107. https://doi.org/10.1109/TIT.1953.1188572

Stark, C., Breitkreutz, B.-J., Reguly, T., Boucher, L., Breitkreutz, A., & Tyers, M. (2006). BioGRID: A general repository for interaction datasets. *Nucleic Acids Research*, *34*(suppl_1), D535–D539. https://doi.org/10.1093/nar/gkj109

Suchanek, F. M., Kasneci, G., & Weikum, G. (2007). YAGO: A Core of Semantic Knowledge Unifying WordNet and Wikipedia. *Semantic Web*, 10.

Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). PathSim: Meta path-based top-k similarity search in heterogeneous information networks. *In Proc. 2011 Int. Conf. Very Large Data Bases (VLDB'11.*

Sun, Y., Yu, Y., & Han, J. (2009). Ranking-based clustering of heterogeneous information networks with star network schema. *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '09*, 797. https://doi.org/10.1145/1557019.1557107

Tang, Jian, Qu, M., & Mei, Q. (2015). PTE: Predictive Text Embedding through Large-scale Heterogeneous Text Networks. *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '15*, 1165–1174. https://doi.org/10.1145/2783258.2783307

Tang, Jian, Qu, M., Wang, M., Zhang, M., Yan, J., & Mei, Q. (2015). LINE: Large-scale Information Network Embedding. *Proceedings of the 24th International Conference on World Wide Web - WWW '15*, 1067–1077. https://doi.org/10.1145/2736277.2741093

Tang, Jie, Zhang, J., Yao, L., Li, J., Zhang, L., & Su, Z. (2008). *ArnetMiner: Extraction and mining of academic social networks*. 990–998. https://doi.org/10.1145/1401890.1402008

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, *9*(1), 5233. https://doi.org/10.1038/s41598-019-41695-z

Tshitoyan, V., Dagdelen, J., Weston, L., Dunn, A., Rong, Z., Kononova, O., Persson, K. A., Ceder, G., & Jain, A. (2019). Unsupervised word embeddings capture latent knowledge from materials science literature. *Nature*, *571*(7763), 95–98. https://doi.org/10.1038/s41586-019-1335-8

Ugander, J., Karrer, B., Backstrom, L., & Marlow, C. (2011). The Anatomy of the Facebook Social Graph. *ArXiv:1111.4503 [Physics]*. http://arxiv.org/abs/1111.4503

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, \Lukasz, & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 5998–6008.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2017). *Graph Attention Networks*. https://arxiv.org/abs/1710.10903v3

Vrandečić, D., & Krötzsch, M. (2014). Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, *57*(10), 78–85. https://doi.org/10.1145/2629489

Wang, Q., Mao, Z., Wang, B., & Guo, L. (2017). Knowledge Graph Embedding: A Survey of Approaches and Applications. *IEEE Transactions on Knowledge and Data Engineering*, *29*(12), 2724–2743. https://doi.org/10.1109/TKDE.2017.2754499

Wang, Z., Zhang, J., Feng, J., & Chen, Z. (2014). Knowledge graph embedding by translating on hyperplanes. *Twenty-Eighth AAAI Conference on Artificial Intelligence*.

Wilcke, X., Bloem, P., & de Boer, V. (2017). The knowledge graph as the default data model for learning on heterogeneous knowledge. *Data Science*, *1*(1–2), 39–57. https://doi.org/10.3233/DS-170007

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2019). A Comprehensive Survey on Graph Neural Networks. *ArXiv:1901.00596 [Cs, Stat]*. http://arxiv.org/abs/1901.00596

Yang, B., Yih, W., He, X., Gao, J., & Deng, L. (2015). Embedding Entities and Relations for Learning and Inference in Knowledge Bases. *ArXiv:1412.6575 [Cs]*. http://arxiv.org/abs/1412.6575

Yang, C., Xiao, Y., Zhang, Y., Sun, Y., & Han, J. (2020). Heterogeneous Network Representation Learning: Survey, Benchmark, Evaluation, and Beyond. *ArXiv:2004.00216 [Cs]*. http://arxiv.org/abs/2004.00216

Yun, S., Jeong, M., Kim, R., Kang, J., & Kim, H. J. (2020). Graph Transformer Networks. *ArXiv:1911.06455 [Cs, Stat]*. http://arxiv.org/abs/1911.06455

Zins, C. (2007). Conceptual approaches for defining data, information, and knowledge. *Journal of the American Society for Information Science and Technology*, *58*(4), 479–493. https://doi.org/10.1002/asi.20508